Assignment 3

Advanced NLP: Monsoon 2026

Deadline: 13th November 2026, 23:59

The range of grade for this assignment is -7% to +7%

General Instructions

1. This assignment must be implemented in Python using Pytorch. You are required to implement the models **from scratch** (so no `mamba.ssm` or any other libraries that abstract away the core implementation of the model).
2. You are allowed to use AI tools or any internet resources as long as your implementation is in accordance with the assignment guidelines. You are **not allowed to use AI to generate the report**. You are allowed to discuss and share code as well, **but not the report**.
3. Your grade will depend on the correctness of your implementation, the quality of your code, the validity of your output, your analysis of the results and methods used, and the clarity of your explanations. Please note that your code and models will be tested on a held out set. Just submitting the deliverables does not guarantee a positive grade.
4. You are required to push the assignment to your assigned `code.iiit.ac.in` repository, and submit your report on Moodle. You are also required to maintain WandB logs and push your models to HuggingFace. These links must be added to your report.
5. Make sure to test your code thoroughly before submission to avoid run-time errors of unexpected behaviour.
6. All clarifications must be requested before **11th November 2026, 23:59**. No clarifications will be entertained after this. We therefore suggest you start the assignment before this deadline. Any assumptions made after this, if unreasonable, shall be penalised accordingly.
7. Editing the assignment pdf or any other such shenanigans shall result in penalties.
8. Incurring multiple penalties can push the marks for this assignment to negative, which will NOT be capped at zero. Thus, it is recommended to adhere to the guidelines.
9. Questions may be asked from the references given at the end of the assignment, so a thorough reading is expected.
10. If any specific case lies outside the instructions, the TAs reserve the right to penalise the same to their discretion.
11. Your code should not have any comments or docstrings.

The assignment has been changed and made easier to account for the delay in its release.

Introduction

Click here to skip the story to the main assignment

For decades, two galactic powers have been locked in a silent but devastating war of intelligence: **Bisleri's Collective**, a decentralized alliance of planetary systems, and the expansionist **Judhanva's Dominion**.

While their fleets clash in asteroid fields and orbital corridors, the true battle is fought through encrypted transmissions carrying fleet movements, supply routes, and strategic commands transmitted across light-years.

Previously *(in INLP)*, the Collective intercepted a high-priority Dominion communication stream. As its cryptographic analyst, you built neural sequence models to learn the cipher and decrypt the intercepted messages. For a while, the Collective was one step ahead.

However, the Dominion caught on. It has since updated its cipher protocol, rendering your previous models ineffective. The Collective has intercepted fresh transmissions, stored in `cipher.jsonl`, along with corresponding plaintext in `cleaned.jsonl`.

As a cryptographic analyst for Bisleri's Collective, you have been called upon once again to build stronger models to learn the underlying cipher and decrypt previously unseen transmissions.

Amidst the war effort, the Dominion has also started spreading false narrative to sow dissent within the ranks of the Collective. You have thus also been tasked with the objective of understanding the propagation of the same; for the Collective to predict and counter the spread and unite the front again.

Implementation Tasks

Part A

Transformers are currently one of the most widely used architectures, especially for language modelling. However, its quadratic time complexity has motivated the development of alternative architectures that aim to achieve comparable performance while scaling linearly with sequence length. Two of the notable ones are Mamba and DeltaNet.

You have been given a dataset consisting of plaintext sequences (`cleaned.jsonl`) and their corresponding encrypted sequences (`cipher.jsonl`). The dataset can be found here. The password for the same is mentioned in the assignment announcement.

In this section, we will explore how well these architectures can learn a cipher function, whether combining them offers any advantage over using them individually, and how well do they compare against a Transformer.

Your mission, should you choose to accept it (you don't have a choice), is to build models that learn the underlying cipher function and decrypt previously unseen encrypted sequences. The train-validation-test split has already been provided.

You are required to implement the following models:
1. **Transformer** (you may reuse your previous implementations)
2. **Mamba** (as described in the paper)
3. **DeltaNet** (as described in the paper)
4. **Mamba-DeltaNet Hybrid**: Design a model composed of Mamba and DeltaNet layers, arranged in any order of your choice (interleaved, sequential, or otherwise). Hypothesise why your chosen arrangement should work, and explain your reasoning in `docs/DESIGN.md`. You must commit and push this file before starting the implementation. The file must not be modified once implementation begins. Your final report must analyse your results with respect to your original hypothesis.

To evaluate and compare your models, report the following metrics/quantities:
1. Character-level accuracy

2. Word-level accuracy
3. Precision
4. Recall
5. F1-Score
6. Leveshtein Distance
7. Parameter count
8. FLOPs

You may use other metrics in addition to the above specified to aid your analysis.

You are allowed to use the concepts covered throughout the course, as long as it adheres to the expected implementation of the alternative architectures (do not use attention interleaved in Mamba, for example).

Your code will be tested on a held-out set. Ensure your code has the ability to pull your own models from HuggingFace and run them, this setting should be on by default during your final push for the submission. The code (for inference) should be able to run via the command `bash scripts/part1/test_assn.sh`. For training, the command should be `bash scripts/part1/test_training.sh`. For more details, refer to the `CLARIFICATIONS.md` file.

Memory Constraints: All models should be able to run inference within 10GB VRAM. Failure to adhere to this constraint will result in loss of marks.

Bonus: You may implement `CUDA` or `Triton` optimizations for this section to speedup the computation. Please note that this is not compulsory. However, if you implement the same and are unable to explain your code and understanding of the implementation, you will be penalised.

Part B

A news propagation graph is a network model that maps how a news story or post spreads across social media.

You will use the GossipCop subset of UPFD. In this dataset, each sample is a news propagation graph where node 0 is the source news post, the remaining nodes are Twitter users who interacted with it. The edges connect these nodes to map out the flow of a tweet between the users. Use PyG's loader to download and cache the data. The train-validation-test split is fixed at 20%-10%-70%; do not change or re-split it.

Architecture Comparison

Train and evaluate the following models using `bert` features (already given in the dataset), with 5 random seeds each:

1. Root-Only MLP: Use only node 0 (the source post) for classification, discarding all retweeter nodes and edges. This serves as your text-only baseline.
2. GCN
3. GraphSAGE
4. GAT
5. Bi-GCN

Compare these models to determine whether the propagation graph improves classification over the source text alone, and which architecture works best.

Feature Ablation

Fix the backbone to GraphSAGE and evaluate it with `profile`, `spacy`, `bert`, and `content` features, using 5 random seeds each. Only the input feature type should change between configurations; keep everything else fixed. Analyse which features contribute most to performance.

Evaluation and Deliverables

For every model/feature configuration in both experiments, report the test-set mean $\pm$ standard deviation across the 5 seeds for:

1. Accuracy
2. Precision
3. Recall
4. F1-Score

For each run, use the checkpoint from the epoch with the best validation F1. Your report must compare all systems and explain your hypotheses for the observed results.

Fill in `results_template.json` with the mean values only and save it as `results.json` inside `output/part2`. For this section, also provide an entry point runnable as `bash ./scripts/part2/test_gnn.sh"`. It must generate the results JSON in the specified output directory. For our testing, the command should be `bash ./scripts/part2/test_gnn.sh --ta` which will output the file as `results_ta.json`.

Submission Instructions

Your report must include a brief overview of the methodology implemented, the configuration(s) used, the results obtained, and your interpretation of the same. Ensure your analysis (and report) is concise and to the point, and that you have provided sufficient evidence (and/or references) to support your claims. You may include tables, plots, diagrams, or any other visual aids to help defend your claims. The report must be in `PDF` format and must be named `<rollnumber>_a3.pdf`. It should not exceed 4 pages (excluding references and link to your HuggingFace models and WandB logs) and should follow the following format:

- Page size: A4, Single column format
- Font size: 12pt (Image captions can be not less than 8pt)
- Font: Times New Roman (or other equivalent sized font. Do not use a smaller font to fit more content, larger are fine if you wish)
- Margins: Not less than 2.5cm on all sides.

Your code must use `uv` to maintain the python environments and run the codes, and the same should be reproducible on `ada` (you may share the `pyproject.toml` file to ensure this). Adhere to the directory structure given in your repository on `code.iiit.ac.in`.

Unnecessary verbosity shall be penalised, so refrain from unnecessary slop. **Do not submit Jupyter notebooks, your assignment will not be graded**.

References

1. State Space Models
2. Implementation Guide for S4 Models
3. Mamba
4. DeltaNet
5. BERT

6. GCN
7. GraphSAGE
8. GAT
9. Bi-GCN